

# API Documentation

Complete reference for integrating OTP verification into your application.

## Authenticator API Documentation

**Version:** 5.0.0 · **Last Updated:** July 2025

**Base URL:** <https://asia-southeast1-authenticator-15fb7.cloudfunctions.net>

**Dashboard:** <https://authenticator-15fb7.web.app>

**Support:** [Contact us through the dashboard]

## Table of Contents

1. [Overview](#)
2. [Quick Start](#)
3. [Authentication](#)
4. [Key Details](#)
5. [API Endpoints](#)
  - [Send OTP](#)
  - [Verify OTP](#)
  - [Check OTP Status / Resend](#)
6. [Error Codes](#)
7. [Rate Limits](#)
8. [Security Best Practices](#)
9. [Integration Guide](#)
10. [Code Examples](#)
  - [Node.js](#)
  - [React + Backend Proxy](#)



- [Next.js API Route](#)
- [Laravel \(PHP\)](#)
- [WordPress](#)
- [Python \(Flask/FastAPI\)](#)
- [Dart \(Server-Side\)](#)

11. [WordPress Plugin](#)

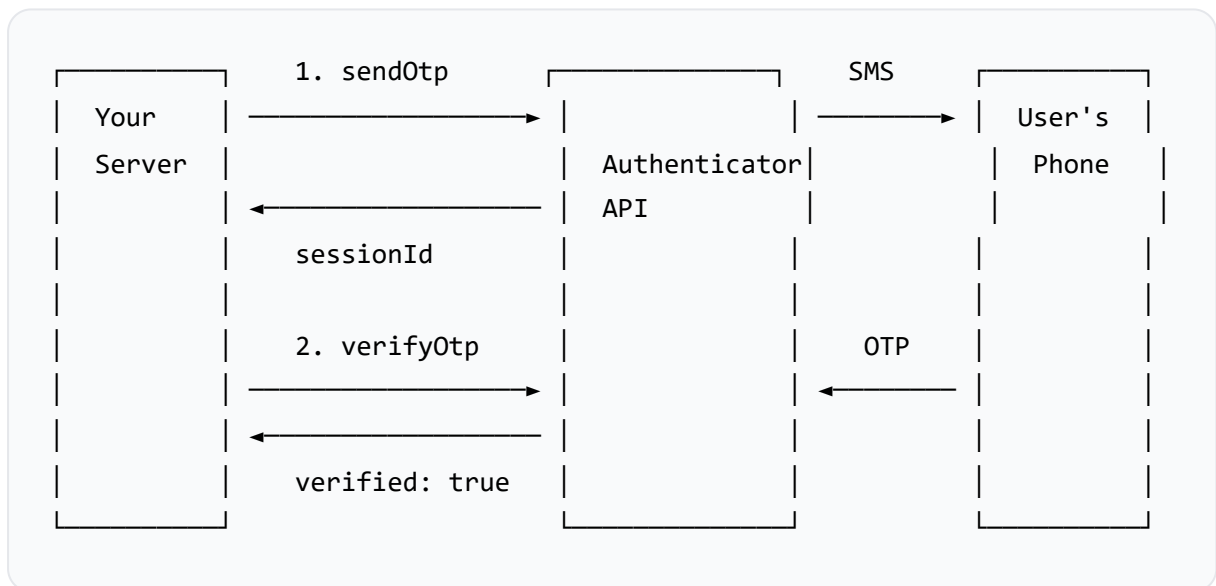
12. [Webhooks](#)

13. [FAQ](#)

## Overview

Authenticator is a production-grade SMS OTP (One-Time Password) verification service. You send a verification code to any mobile number, and we handle the rest — SMS delivery, OTP generation, expiry, and verification.

### How it works:



## Quick Start

Get up and running in under 5 minutes:

### Step 1: Register Your App

Go to <https://authenticator-15fb7.web.app> and create a new app. You will receive:

- **appId** — your public app identifier
- **appSecret** — your private app secret (shown once — store it securely)

## Step 2: Buy Credits

Purchase an SMS credit package from the dashboard. Each `sendOtp` call costs 1 credit.

## Step 3: Send Your First OTP

```
curl -X POST \
  https://asia-southeast1-authenticator-15fb7.cloudfunctions.net/sendOtp `
-H "Content-Type: application/json" \
-d '{
  "appId": "your-app-id",
  "appSecret": "your-app-secret",
  "phoneNumber": "+8801712345678"
}'
```

## Step 4: Verify the OTP

```
curl -X POST \
  https://asia-southeast1-authenticator-15fb7.cloudfunctions.net/verifyOtp `
-H "Content-Type: application/json" \
-d '{
  "appId": "your-app-id",
  "sessionId": "the-session-id-from-step-3",
  "otp": "123456"
}'
```

# Authentication

All API requests are authenticated using **app credentials** passed in the JSON request body:

| FIELD     | DESCRIPTION                | WHERE TO FIND                                     |
|-----------|----------------------------|---------------------------------------------------|
| appId     | Your public app identifier | Dashboard → App Settings                          |
| appSecret | Your private app secret    | Dashboard → App Settings (shown once at creation) |

 **CRITICAL:** Never expose appSecret in client-side code (JavaScript, mobile apps, etc.). Always call the API from your **backend server**. See [Security Best Practices](#).

## Key Details

| FEATURE                   | DETAILS                                                                              |
|---------------------------|--------------------------------------------------------------------------------------|
| Supported Countries       | Bangladesh (+880) · More countries available on request                              |
| OTP Length                | 6 digits (e.g., 384726)                                                              |
| OTP Expiry                | 10 minutes from send time                                                            |
| Max Verification Attempts | 3 attempts per session (locks after 3 wrong guesses)                                 |
| Rate Limit (per phone)    | 3 OTPs per 10 minutes (configurable per app)                                         |
| Rate Limit (cooldown)     | 30-second cooldown between OTPs to the same number                                   |
| Max Resends               | 2 resends per session                                                                |
| SMS Template              | Customizable per app: Your {appName} code: {otp}. Valid {ttl} minutes. Do not share. |
| Request Format            | JSON body (Content-Type: application/json)                                           |

| FEATURE             | DETAILS                                                                                            |
|---------------------|----------------------------------------------------------------------------------------------------|
| Response Format     | JSON (application/json)                                                                            |
| Protocol            | HTTPS only (TLS 1.2+)                                                                              |
| Test/Sandbox Mode   | Use production endpoints with a test phone number.<br>Purchase a small credit package for testing. |
| Phone Number Format | E.164 format required (e.g., +8801712345678)                                                       |

## SMS Template

You can customize the SMS message template in your app settings on the dashboard. The following placeholders are supported:

| PLACEHOLDER | REPLACED WITH                                   |
|-------------|-------------------------------------------------|
| {appName}   | Your app's display name                         |
| {otp}       | The 6-digit OTP code                            |
| {ttl}       | The OTP validity duration in minutes (e.g., 10) |

### Default template:

Your {appName} code: {otp}. Valid {ttl} minutes. Do not share.

### Custom example:

Your HaatBazar verification code is {otp}. This code expires in {ttl} minute

## API Endpoints

### 1. Send OTP

Send a 6-digit OTP to a mobile phone number.

POST /send0tp

**Base URL:** `https://asia-southeast1-authenticator-15fb7.cloudfunctions.net/send0tp`

### Request Body

| FIELD       | TYPE   | REQUIRED | DESCRIPTION                                         |
|-------------|--------|----------|-----------------------------------------------------|
| appId       | string | ✓        | Your app identifier                                 |
| appSecret   | string | ✓        | Your app secret                                     |
| phoneNumber | string | ✓        | Phone number in E.164 format (e.g., +8801712345678) |

### Request Example

```
{
  "appId": "app_abc123",
  "appSecret": "secret_xyz789",
  "phoneNumber": "+8801712345678"
}
```

### Success Response — 200 OK

```
{
  "sessionId": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
  "expiresAt": 1720000000000
}
```

| FIELD     | TYPE   | DESCRIPTION                                                                  |
|-----------|--------|------------------------------------------------------------------------------|
| sessionId | string | Unique session identifier (UUID). Store this — you need it for verification. |
| expiresAt | number | Unix timestamp (milliseconds) when the OTP expires.                          |

## Response Headers

| HEADER        | DESCRIPTION                                 |
|---------------|---------------------------------------------|
| X-Request-ID  | Unique request identifier for debugging     |
| Server-Timing | Processing time in milliseconds             |
| Retry-After   | Seconds until retry (only on 429 responses) |

## Error Responses

| STATUS | ERROR               | DESCRIPTION                                            |
|--------|---------------------|--------------------------------------------------------|
| 400    | bad_request         | Missing required fields or invalid phone number format |
| 402    | no_credits          | No credit package purchased or SMS credits depleted    |
| 402    | credits_expired     | Credit package has expired                             |
| 403    | app_not_found       | Invalid appId                                          |
| 403    | invalid_credentials | Wrong appSecret                                        |
| 403    | app_revoked         | App has been revoked/suspended                         |
| 429    | rate_limited        | Too many OTP requests for this phone number            |
| 429    | cooldown            | 30-second cooldown not elapsed for this phone number   |
| 503    | service_paused      | SMS sending is temporarily paused (maintenance)        |
| 500    | integrity_error     | Internal server error                                  |

## Error Response Example

```
{  
  "error": "rate_limited",
```

```
"message": "Too many OTP requests. Please try again later."
}
```

## 2. Verify OTP

Verify the OTP code entered by the user.

POST /verifyOtp

**Base URL:** <https://asia-southeast1-authenticator-15fb7.cloudfunctions.net/verifyOtp>

### Request Body

| FIELD     | TYPE   | REQUIRED | DESCRIPTION                          |
|-----------|--------|----------|--------------------------------------|
| appId     | string | ✓        | Your app identifier                  |
| sessionId | string | ✓        | Session ID returned by sendOtp       |
| otp       | string | ✓        | 6-digit OTP code entered by the user |

### Request Example

```
{
  "appId": "app_abc123",
  "sessionId": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
  "otp": "384726"
}
```

### Success Response — 200 OK (Verified)

```
{
  "verified": true,
  "phoneNumber": "+8801712345678"
}
```



| FIELD       | TYPE    | DESCRIPTION                               |
|-------------|---------|-------------------------------------------|
| verified    | boolean | true if the OTP matches                   |
| phoneNumber | string  | The verified phone number in E.164 format |

### Failure Response — 200 OK (Not Verified)

```
{
  "verified": false,
  "reason": "mismatch"
}
```

| REASON VALUE | DESCRIPTION                            |
|--------------|----------------------------------------|
| mismatch     | OTP does not match                     |
| expired      | OTP has expired                        |
| locked       | Session locked after 3 failed attempts |
| not_found    | Session ID does not exist              |

### Error Responses

| STATUS | ERROR           | DESCRIPTION                            |
|--------|-----------------|----------------------------------------|
| 400    | bad_request     | Missing fields or OTP is not 6 digits  |
| 403    | —               | appId does not match the session's app |
| 404    | —               | Session not found                      |
| 410    | —               | OTP has expired                        |
| 423    | —               | Session locked (3 failed attempts)     |
| 500    | integrity_error | Internal server error                  |

### 3. Check OTP Status / Resend

Check the delivery status of an OTP. Optionally trigger a **resend** if the original SMS failed to deliver.

POST /otpStatus

**Base URL:** `https://asia-southeast1-authenticator-15fb7.cloudfunctions.net/otpStatus`

**Note:** This is NOT a separate "resend OTP" endpoint. It checks the status of a session and, if `resend: true` is passed and the SMS failed, generates a new OTP and re-queues the SMS.

#### Request Body

| FIELD     | TYPE    | REQUIRED | DESCRIPTION                                      |
|-----------|---------|----------|--------------------------------------------------|
| appId     | string  | ✓        | Your app identifier                              |
| sessionId | string  | ✓        | Session ID returned by sendOtp                   |
| resend    | boolean | ✗        | Set to true to resend if the original SMS failed |

#### Request Example — Check Status

```
{
  "appId": "app_abc123",
  "sessionId": "f47ac10b-58cc-4372-a567-0e02b2c3d479"
}
```

#### Request Example — Resend on Failure

```
{
  "appId": "app_abc123",
  "sessionId": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
  "resend": true
}
```

## Responses

The response varies based on the current status:

status: "pending"

SMS is queued and waiting to be sent by the authenticator device.

```
{  
  "status": "pending"  
}
```

status: "sent"

SMS was successfully delivered. Includes the delivery timestamp.

```
{  
  "status": "sent",  
  "sent_at": 1720000000000,  
  "message": "OTP was already confirmed delivered to this number."  
}
```

status: "expired"

The OTP session has expired.

```
{  
  "status": "expired"  
}
```

status: "failed"

SMS delivery failed.

```
{  
  "status": "failed",  
  "error": "SMS send failed"  
}
```

status: "not\_found"

Session ID does not exist (never created or already verified and cleaned up).

```
{
  "status": "not_found"
}
```

Resend Response — 200 OK

When resend: true is passed and the resend succeeds:

```
{
  "status": "pending",
  "sessionId": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
  "expiresAt": 1720000600000
}
```

Resend Error Responses

| STATUS | ERROR                | DESCRIPTION                                          |
|--------|----------------------|------------------------------------------------------|
| 429    | max_resends_exceeded | Maximum 2 resends per session reached                |
| 429    | cooldown             | 30-second cooldown not elapsed for this phone number |
| 503    | service_paused       | SMS sending is temporarily paused                    |

Error Codes

Complete Error Reference

| HTTP STATUS | ERROR CODE  | DESCRIPTION                   | RESOLUTION                      |
|-------------|-------------|-------------------------------|---------------------------------|
| 400         | bad_request | Missing or invalid parameters | Check request body fields       |
| 402         | no_credits  | No SMS credits remaining      | Purchase credits from dashboard |

| HTTP STATUS | ERROR CODE           | DESCRIPTION                     | RESOLUTION                                                |
|-------------|----------------------|---------------------------------|-----------------------------------------------------------|
| 402         | credits_expired      | Credit package expired          | Purchase a new credit package                             |
| 403         | app_not_found        | Invalid appId                   | Verify your appId from the dashboard                      |
| 403         | invalid_credentials  | Wrong appSecret                 | Verify your appSecret from the dashboard                  |
| 403         | app_revoked          | App suspended                   | Contact support                                           |
| 403         | app_mismatch         | appId doesn't match the session | Use the correct appId for this session                    |
| 404         | —                    | Session not found               | The session may have expired or been verified already     |
| 405         | method_not_allowed   | Wrong HTTP method               | Use POST only                                             |
| 410         | —                    | OTP expired                     | Request a new OTP                                         |
| 423         | —                    | Session locked                  | 3 failed verify attempts — request a new OTP              |
| 429         | rate_limited         | Rate limit exceeded             | Wait and retry (see Retry-After header)                   |
| 429         | cooldown             | Phone number cooldown           | Wait (see waitSeconds in response)                        |
| 429         | max_resends_exceeded | Too many resends                | Max 2 resends per session — request a new OTP via sendOtp |
|             |                      |                                 |                                                           |

| HTTP STATUS | ERROR CODE      | DESCRIPTION      | RESOLUTION           |
|-------------|-----------------|------------------|----------------------|
| 500         | integrity_error | Internal error   | Retry the request    |
| 503         | service_paused  | Maintenance mode | Wait and retry later |

## Rate Limits

### Per-Phone Rate Limit

- **Default:** 3 OTP requests per phone number per 10 minutes
- Configurable per app (contact support to increase)
- **Cooldown:** 30 seconds between OTPs to the same phone number (enforced server-side)

### Rate Limit Headers

| HEADER      | EXAMPLE | DESCRIPTION                                         |
|-------------|---------|-----------------------------------------------------|
| Retry-After | 120     | Seconds until you can retry (only on 429 responses) |

### Rate Limit Response

```
{
  "error": "rate_limited",
  "message": "Too many OTP requests. Please try again later."
}
```

### Best Practices for Rate Limiting

1. **Show a countdown timer** to the user after sending an OTP
2. **Disable the "Resend" button** for 30 seconds after sending
3. **Cache the sessionId** — don't create a new session if one is still valid
4. **Use otpStatus with resend: true** instead of calling sendOtp again for the same number

# Security Best Practices

## ● Critical Rules

1. **NEVER embed appSecret in client-side code.** This includes:

- React/Vue/Angular components
- Mobile app code (Android/iOS)
- Browser localStorage or cookies
- Client-side JavaScript bundles

2. **Always call the API from your backend server.** Use a proxy endpoint:

```
User's Browser → Your Backend API → Authenticator API
                (no secrets)         (has appSecret)
```

3. **Store secrets in environment variables.** Use .env files (never committed to Git) or a secrets manager.

4. **Use HTTPS everywhere.** Never call the API over plain HTTP.

## Environment Variable Setup

```
# .env (never commit this file to version control)
AUTHENTICATOR_APP_ID=app_abc123
AUTHENTICATOR_APP_SECRET=secret_xyz789
AUTHENTICATOR_BASE_URL=https://asia-southeast1-authenticator-15fb7.cloudfi
```

```
# .gitignore
.env
.env.local
.env.production
```

## Verification Flow Security

- The OTP is **6 digits** — brute-force is prevented by the **3-attempt lock** per session
- Sessions expire after **10 minutes** — expired OTPs cannot be verified
- Phone numbers must be in **E.164 format** — prevents format-based bypasses
- **Atomic operations** prevent race conditions during verification

## Integration Guide

### Step-by-Step Integration

#### Step 1: Register Your App

1. Go to <https://authenticator-15fb7.web.app>
2. Sign up / log in
3. Click "**Create New App**"
4. Enter your app name and (optionally) a custom SMS template
5. Save your **appId** and **appSecret** securely

#### Step 2: Purchase Credits



the dashboard, go to "Credits"



2. Select a credit package

3. Complete the purchase

#### Step 3: Add Environment Variables

Add to your server's `.env` file:

```
AUTHENTICATOR_APP_ID=your-app-id-here
AUTHENTICATOR_APP_SECRET=your-app-secret-here
AUTHENTICATOR_BASE_URL=https://asia-southeast1-authenticator-15fb7.cloudfi
```

#### Step 4: Implement the Flow

Typical verification flow:



1. User enters phone number
2. Your frontend calls your backend: `POST /api/auth/send-otp { phone }`
3. Your backend calls Authenticator: `POST /sendOtp { appId, appSecret, phone }`
4. Return `sessionId` to frontend
5. User enters the 6-digit code from SMS
6. Frontend calls your backend: `POST /api/auth/verify-otp { sessionId, otp }`
7. Your backend calls Authenticator: `POST /verifyOtp { appId, sessionId, otp }`
8. Return `verified: true/false` to frontend
9. If verified → create session / issue token

## Step 5: Test

- Use a real Bangladesh (+880) phone number
- Verify you receive the SMS
- Test the full send → verify → success flow
- Test error cases: wrong OTP, expired OTP, rate limiting

# Code Examples

## Node.js

⚠ **FLAG:** This code runs on your **backend server** only. Never expose `appSecret` in frontend code.

## Using fetch (Node.js 18+)

```
// services/authenticator.js

const AUTHENTICATOR_BASE_URL = process.env.AUTHENTICATOR_BASE_URL;
const AUTHENTICATOR_APP_ID = process.env.AUTHENTICATOR_APP_ID;
const AUTHENTICATOR_APP_SECRET = process.env.AUTHENTICATOR_APP_SECRET;

/**
 * Send an OTP to a phone number.
 * @param {string} phoneNumber - Phone number in E.164 format (e.g., +880:
 * @returns {Promise<{sessionId: string, expiresAt: number}>}
 */
```

```
async function sendOtp(phoneNumber) {
  const response = await fetch(`${AUTHENTICATOR_BASE_URL}/sendOtp`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      appId: AUTHENTICATOR_APP_ID,
      appSecret: AUTHENTICATOR_APP_SECRET,
      phoneNumber,
    }),
  });

  const data = await response.json();

  if (!response.ok) {
    throw new Error(data.message || data.error || 'Failed to send OTP');
  }

  return data;
}

/**
 * Verify an OTP code.
 * @param {string} sessionId - Session ID from sendOtp
 * @param {string} otp - 6-digit OTP code
 * @returns {Promise<{verified: boolean, phoneNumber?: string, reason?: string}>}
 */
async function verifyOtp(sessionId, otp) {
  const response = await fetch(`${AUTHENTICATOR_BASE_URL}/verifyOtp`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      appId: AUTHENTICATOR_APP_ID,
      sessionId,
      otp,
    }),
  });

  return response.json();
}

/**
 * Check OTP delivery status or resend on failure.
 * @param {string} sessionId - Session ID from sendOtp
 * @param {boolean} resend - Whether to resend if SMS failed
 * @returns {Promise<{status: string, expiresAt?: number}>}
 */
```

```

async function checkOtpStatus(sessionId, resend = false) {
  const response = await fetch(`${AUTHENTICATOR_BASE_URL}/otpStatus`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      appId: AUTHENTICATOR_APP_ID,
      sessionId,
      resend,
    }),
  });

  return response.json();
}

module.exports = { sendOtp, verifyOtp, checkOtpStatus };

```

## Using axios

```

// services/authenticator.js

const axios = require('axios');

const client = axios.create({
  baseURL: process.env.AUTHENTICATOR_BASE_URL,
  headers: { 'Content-Type': 'application/json' },
});

const APP_ID = process.env.AUTHENTICATOR_APP_ID;
const APP_SECRET = process.env.AUTHENTICATOR_APP_SECRET;

/**
 * Send an OTP to a phone number.
 * @param {string} phoneNumber - E.164 format
 * @returns {Promise<{sessionId: string, expiresAt: number}>}
 */
async function sendOtp(phoneNumber) {
  const { data } = await client.post('/sendOtp', {
    appId: APP_ID,
    appSecret: APP_SECRET,
    phoneNumber,
  });
  return data;
}

```

```

/**
 * Verify an OTP code.
 * @param {string} sessionId - Session ID from sendOtp
 * @param {string} otp - 6-digit code
 * @returns {Promise<{verified: boolean, phoneNumber?: string}>}
 */
async function verifyOtp(sessionId, otp) {
  const { data } = await client.post('/verifyOtp', {
    appId: APP_ID,
    sessionId,
    otp,
  });
  return data;
}

module.exports = { sendOtp, verifyOtp };

```

## Express.js Route Example

```

// routes/auth.js

const express = require('express');
const router = express.Router();
const { sendOtp, verifyOtp } = require('../services/authenticator');

/**
 * POST /api/auth/send-otp
 * Body: { phoneNumber: "+8801712345678" }
 */
router.post('/send-otp', async (req, res) => {
  try {
    const { phoneNumber } = req.body;

    if (!phoneNumber) {
      return res.status(400).json({ error: 'Phone number is required' });
    }

    const result = await sendOtp(phoneNumber);
    return res.json(result);
  } catch (error) {
    return res.status(500).json({ error: error.message });
  }
});

```

```
/**
 * POST /api/auth/verify-otp
 * Body: { sessionId: "...", otp: "123456" }
 */
router.post('/verify-otp', async (req, res) => {
  try {
    const { sessionId, otp } = req.body;

    if (!sessionId || !otp) {
      return res.status(400).json({ error: 'sessionId and otp are required' });
    }

    const result = await verifyOtp(sessionId, otp);

    if (result.verified) {
      // OTP verified – create user session, issue JWT, etc.
      // const token = createSession(result.phoneNumber);
      return res.json({ success: true, phoneNumber: result.phoneNumber });
    }

    return res.status(400).json({
      success: false,
      reason: result.reason,
    });
  } catch (error) {
    return res.status(500).json({ error: error.message });
  }
});

module.exports = router;
```

## React + Backend Proxy

 **CRITICAL:** Never call the Authenticator API directly from React. Always proxy through your backend.

## Frontend (React)

```
// hooks/useOtp.js

import { useState } from 'react';

/**
 * Custom hook for OTP verification flow.
 * Calls YOUR backend, not the Authenticator API directly.
 */
export function useOtp() {
  const [loading, setLoading] = useState(false);
  const [sessionId, setSessionId] = useState(null);
  const [error, setError] = useState(null);

  /**
   * Send OTP to a phone number via your backend proxy.
   * @param {string} phoneNumber - E.164 format
   */
  const sendCode = async (phoneNumber) => {
    setLoading(true);
    setError(null);

    try {
      const response = await fetch('/api/auth/send-otp', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ phoneNumber }),
      });

      const data = await response.json();

      if (!response.ok) {
        throw new Error(data.error || 'Failed to send OTP');
      }

      setSessionId(data.sessionId);
      return data;
    } catch (err) {
      setError(err.message);
      throw err;
    } finally {
      setLoading(false);
    }
  };

  /**
```

```
* Verify the OTP code via your backend proxy.
* @param {string} otp - 6-digit code entered by the user
*/
const verifyCode = async (otp) => {
  if (!sessionId) {
    throw new Error('No active session. Send an OTP first.');
```

```
  }

  setLoading(true);
  setError(null);

  try {
    const response = await fetch('/api/auth/verify-otp', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ sessionId, otp }),
    });

    const data = await response.json();

    if (!response.ok) {
      throw new Error(data.error || 'Verification failed');
    }

    return data;
  } catch (err) {
    setError(err.message);
    throw err;
  } finally {
    setLoading(false);
  }
};

return { sendCode, verifyCode, loading, error, sessionId };
```

```
// components/OtpForm.jsx

import React, { useState } from 'react';
import { useOtp } from '../hooks/useOtp';

export function OtpForm() {
  const [phone, setPhone] = useState('');
  const [otp, setOtp] = useState('');
```

```
const [step, setStep] = useState('phone'); // 'phone' | 'verify'
const [verified, setVerified] = useState(false);
const { sendCode, verifyCode, loading, error } = useOtp();

const handleSendOtp = async (e) => {
  e.preventDefault();
  try {
    await sendCode(phone);
    setStep('verify');
  } catch {
    // Error is already set in the hook
  }
};

const handleVerifyOtp = async (e) => {
  e.preventDefault();
  try {
    const result = await verifyCode(otp);
    if (result.success) {
      setVerified(true);
    }
  } catch {
    // Error is already set in the hook
  }
};

if (verified) {
  return <div>Phone number verified successfully!</div>;
}

if (step === 'verify') {
  return (
    <form onSubmit={handleVerifyOtp}>
      <h2>Enter Verification Code</h2>
      <p>We sent a 6-digit code to {phone}</p>
      <input
        type="text"
        value={otp}
        onChange={(e) => setOtp(e.target.value)}
        placeholder="123456"
        maxLength={6}
        pattern="\d{6}"
        required
      />
      <button type="submit" disabled={loading}>
        {loading ? 'Verifying...' : 'Verify'}
      </button>
    </form>
  );
}
```



```

        </button>
        {error && <p className="error">{error}</p>}
    </form>
  );
}

return (
  <form onSubmit={handleSendOtp}>
    <h2>Verify Your Phone</h2>
    <input
      type="tel"
      value={phone}
      onChange={(e) => setPhone(e.target.value)}
      placeholder="+8801712345678"
      required
    />
    <button type="submit" disabled={loading}>
      {loading ? 'Sending...' : 'Send Code'}
    </button>
    {error && <p className="error">{error}</p>}
  </form>
);
}

```

## Next.js API Route

 **FLAG:** appSecret is server-side only. Next.js API routes run on the server and are safe for secrets.

### app/api/auth/send-otp/route.ts

```

import { NextRequest, NextResponse } from 'next/server';

const BASE_URL = process.env.AUTHENTICATOR_BASE_URL!;
const APP_ID = process.env.AUTHENTICATOR_APP_ID!;
const APP_SECRET = process.env.AUTHENTICATOR_APP_SECRET!;

export async function POST(request: NextRequest) {
  try {
    const { phoneNumber } = await request.json();

```

```
if (!phoneNumber) {
  return NextResponse.json(
    { error: 'Phone number is required' },
    { status: 400 },
  );
}

const response = await fetch(`${BASE_URL}/sendOtp`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    appId: APP_ID,
    appSecret: APP_SECRET,
    phoneNumber,
  }),
});

const data = await response.json();

if (!response.ok) {
  return NextResponse.json(
    { error: data.message || data.error },
    { status: response.status },
  );
}

return NextResponse.json(data);
} catch (error) {
  return NextResponse.json(
    { error: 'Internal server error' },
    { status: 500 },
  );
}
```

#### app/api/auth/verify-otp/route.ts

```
import { NextRequest, NextResponse } from 'next/server';

const BASE_URL = process.env.AUTHENTICATOR_BASE_URL!;
const APP_ID = process.env.AUTHENTICATOR_APP_ID!;
const APP_SECRET = process.env.AUTHENTICATOR_APP_SECRET!;
```

```
export async function POST(request: NextRequest) {
  try {
    const { sessionId, otp } = await request.json();

    if (!sessionId || !otp) {
      return NextResponse.json(
        { error: 'sessionId and otp are required' },
        { status: 400 },
      );
    }

    const response = await fetch(`${BASE_URL}/verifyOtp`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        appId: APP_ID,
        appSecret: APP_SECRET,
        sessionId,
        otp,
      }),
    });

    const data = await response.json();

    if (data.verified) {
      // Create user session, issue JWT, etc.
      return NextResponse.json({
        success: true,
        phoneNumber: data.phoneNumber,
      });
    }

    return NextResponse.json(
      { success: false, reason: data.reason },
      { status: 200 },
    );
  } catch (error) {
    return NextResponse.json(
      { error: 'Internal server error' },
      { status: 500 },
    );
  }
}
```

```
// app/login/page.tsx

'use client';

import { useState } from 'react';

export default function LoginPage() {
  const [phone, setPhone] = useState('');
  const [otp, setOtp] = useState('');
  const [step, setStep] = useState<'phone' | 'verify'>('phone');
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');

  const handleSend = async () => {
    setLoading(true);
    setError('');
    try {
      const res = await fetch('/api/auth/send-otp', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ phoneNumber: phone }),
      });
      const data = await res.json();
      if (!res.ok) throw new Error(data.error);
      setStep('verify');
    } catch (err: unknown) {
      setError(err instanceof Error ? err.message : 'Failed to send OTP')
    } finally {
      setLoading(false);
    }
  };

  const handleVerify = async () => {
    setLoading(true);
    setError('');
    try {
      const res = await fetch('/api/auth/verify-otp', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ sessionId: '', otp }), // sessionId from s
      });
      const data = await res.json();
      if (data.success) {
        // Handle successful verification
      } else {

```

```
        setError(data.reason || 'Verification failed');
    }
} catch (err: unknown) {
    setError(err instanceof Error ? err.message : 'Verification failed')
} finally {
    setLoading(false);
}
};

return (
    <div>
        {step === 'phone' ? (
            <div>
                <input
                    type="tel"
                    value={phone}
                    onChange={(e) => setPhone(e.target.value)}
                    placeholder="+8801712345678"
                />
                <button onClick={handleSend} disabled={loading}>
                    Send Code
                </button>
            </div>
        ) : (
            <div>
                <input
                    type="text"
                    value={otp}
                    onChange={(e) => setOtp(e.target.value)}
                    placeholder="123456"
                    maxLength={6}
                />
                <button onClick={handleVerify} disabled={loading}>
                    Verify
                </button>
            </div>
        )}
        {error && <p>{error}</p>}
    </div>
);
}
```

## Laravel (PHP)

⚠ **FLAG:** Requires Laravel 8+ with the HTTP facade. Store credentials in `.env`.

### Configuration

Add to `config/services.php`:

```
// config/services.php

return [
    // ... existing services

    'authenticator' => [
        'base_url' => env('AUTHENTICATOR_BASE_URL'),
        'app_id' => env('AUTHENTICATOR_APP_ID'),
        'app_secret' => env('AUTHENTICATOR_APP_SECRET'),
    ],
];
```

Add to `.env`:

```
AUTHENTICATOR_BASE_URL=https://asia-southeast1-authenticator-15fb7.cloudfi
AUTHENTICATOR_APP_ID=your-app-id
AUTHENTICATOR_APP_SECRET=your-app-secret
```

### Service Class

```
<?php

// app/Services/AuthenticatorService.php

namespace App\Services;

use Illuminate\Http\Client\ConnectionException;
use Illuminate\Support\Facades\Http;
use Illuminate\Support\Facades\Log;

class AuthenticatorService
{
```

```

private string $baseUrl;
private string $appId;
private string $appSecret;

public function __construct()
{
    $this->baseUrl = config('services.authenticator.base_url');
    $this->appId = config('services.authenticator.app_id');
    $this->appSecret = config('services.authenticator.app_secret');
}

/**
 * Send an OTP to a phone number.
 *
 * @param string $phoneNumber Phone number in E.164 format (e.g., +8801712345678)
 * @return array{sessionId: string, expiresAt: int}
 *
 * @throws \Exception
 */
public function sendOtp(string $phoneNumber): array
{
    $response = Http::post("{ $this->baseUrl }/sendOtp", [
        'appId' => $this->appId,
        'appSecret' => $this->appSecret,
        'phoneNumber' => $phoneNumber,
    ]);

    if ($response->failed()) {
        Log::error('Authenticator sendOtp failed', [
            'status' => $response->status(),
            'body' => $response->json(),
        ]);

        throw new \Exception(
            $response->json('message', 'Failed to send OTP')
        );
    }

    return $response->json();
}

/**
 * Verify an OTP code.
 *
 * @param string $sessionId Session ID from sendOtp
 * @param string $otp 6-digit OTP code

```

```

* @return array{verified: bool, phoneNumber?: string, reason?: string}
*
* @throws \Exception
*/
public function verifyOtp(string $sessionId, string $otp): array
{
    $response = Http::post("{ $this->baseUrl }/verifyOtp", [
        'appId' => $this->appId,
        'appSecret' => $this->appSecret,
        'sessionId' => $sessionId,
        'otp' => $otp,
    ]);

    if ($response->failed()) {
        Log::error('Authenticator verifyOtp failed', [
            'status' => $response->status(),
            'body' => $response->json(),
        ]);

        throw new \Exception(
            $response->json('message', 'Failed to verify OTP')
        );
    }

    return $response->json();
}

/**
 * Check OTP status or resend on failure.
 *
 * @param string $sessionId Session ID from sendOtp
 * @param bool $resend Whether to resend if SMS failed
 * @return array{status: string, expiresAt?: int}
 */
public function checkOtpStatus(string $sessionId, bool $resend = false)
{
    $response = Http::post("{ $this->baseUrl }/otpStatus", [
        'appId' => $this->appId,
        'sessionId' => $sessionId,
        'resend' => $resend,
    ]);

    return $response->json();
}
}

```



## Controller

```
<?php

// app/Http/Controllers/OtpController.php

namespace App\Http\Controllers;

use App\Services\AuthenticatorService;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;
use Illuminate\Validation\ValidationException;

class OtpController extends Controller
{
    private AuthenticatorService $authenticator;

    public function __construct(AuthenticatorService $authenticator)
    {
        $this->authenticator = $authenticator;
    }

    /**
     * Send an OTP to the given phone number.
     */
    public function send(Request $request): JsonResponse
    {
        $validated = $request->validate([
            'phone_number' => 'required|string|regex:/^[1-9]\d{1,14}$/'
        ]);

        try {
            $result = $this->authenticator->sendOtp($validated['phone_num

            // Store sessionId in the user's session for later verificati
            session(['otp_session_id' => $result['sessionId']]);

            return response()->json([
                'success' => true,
                'expires_at' => $result['expiresAt'],
            ]);
        } catch (\Exception $e) {
            return response()->json([
```

```
        'success' => false,
        'error' => $e->getMessage(),
    ], 500);
    }
}

/**
 * Verify the OTP code.
 */
public function verify(Request $request): JsonResponse
{
    $validated = $request->validate([
        'otp' => 'required|string|digits:6',
    ]);

    $sessionId = session('otp_session_id');

    if (! $sessionId) {
        throw ValidationException::withMessages([
            'otp' => 'No active OTP session. Please request a new code',
        ]);
    }

    try {
        $result = $this->authenticator->verifyOtp(
            $sessionId,
            $validated['otp']
        );

        if ($result['verified']) {
            // Clear the session
            session()->forget('otp_session_id');

            // Mark phone as verified, create user, issue token, etc.
            return response()->json([
                'success' => true,
                'phone_number' => $result['phoneNumber'],
            ]);
        }

        return response()->json([
            'success' => false,
            'reason' => $result['reason'],
        ]);
    } catch (\Exception $e) {
        return response()->json([
```

```

        'success' => false,
        'error' => $e->getMessage(),
    ], 500);
    }
}
}

```

## Routes

```

// routes/api.php

use App\Http\Controllers\OtpController;

Route::post('/auth/send-otp', [OtpController::class, 'send']);
Route::post('/auth/verify-otp', [OtpController::class, 'verify']);

```

## WordPress

 **FLAG:** This code goes in your theme's functions.php or a custom plugin file. Never put appSecret in a page template.

```

<?php

/**
 * Authenticator OTP Service for WordPress.
 *
 * Add these constants to wp-config.php:
 *
 * define('AUTHENTICATOR_BASE_URL', 'https://asia-southeast1-authenticato
 * define('AUTHENTICATOR_APP_ID', 'your-app-id');
 * define('AUTHENTICATOR_APP_SECRET', 'your-app-secret');
 */

/**
 * Send an OTP to a phone number.
 *
 * @param string $phone_number Phone number in E.164 format (e.g., +88017:
 * @return array|WP_Error

```

```

*/
function authenticator_send_otp($phone_number) {
    $response = wp_remote_post(AUTHENTICATOR_BASE_URL . '/sendOtp', array
        'headers' => array('Content-Type' => 'application/json'),
        'body'     => json_encode(array(
            'appId'      => AUTHENTICATOR_APP_ID,
            'appSecret'  => AUTHENTICATOR_APP_SECRET,
            'phoneNumber' => $phone_number,
        )),
        'timeout' => 30,
    ));

    if (is_wp_error($response)) {
        return $response;
    }

    $body = json_decode(wp_remote_retrieve_body($response), true);
    $code = wp_remote_retrieve_response_code($response);

    if ($code !== 200) {
        return new WP_Error(
            $body['error'] ?? 'unknown_error',
            $body['message'] ?? 'Failed to send OTP',
            array('status' => $code)
        );
    }

    return $body;
}

/**
 * Verify an OTP code.
 *
 * @param string $session_id Session ID from authenticator_send_otp
 * @param string $otp        6-digit OTP code
 * @return array|WP_Error
 */
function authenticator_verify_otp($session_id, $otp) {
    $response = wp_remote_post(AUTHENTICATOR_BASE_URL . '/verifyOtp', array
        'headers' => array('Content-Type' => 'application/json'),
        'body'     => json_encode(array(
            'appId'      => AUTHENTICATOR_APP_ID,
            'appSecret'  => AUTHENTICATOR_APP_SECRET,
            'sessionId'  => $session_id,
            'otp'        => $otp,
        )),

```

```

        'timeout' => 30,
    ));

    if (is_wp_error($response)) {
        return $response;
    }

    $body = json_decode(wp_remote_retrieve_body($response), true);
    $code = wp_remote_retrieve_response_code($response);

    if ($code !== 200) {
        return new WP_Error(
            $body['error'] ?? 'unknown_error',
            $body['message'] ?? 'Failed to verify OTP',
            array('status' => $code)
        );
    }

    return $body;
}

/**
 * AJAX handler: Send OTP.
 * Hook: wp_ajax_nopriv_send_otp, wp_ajax_send_otp
 */
function authenticator_ajax_send_otp() {
    check_ajax_referer('otp_nonce', 'nonce');

    $phone_number = sanitize_text_field($_POST['phone_number'] ?? '');

    if (empty($phone_number)) {
        wp_send_json_error(array('message' => 'Phone number is required'))
    }

    $result = authenticator_send_otp($phone_number);

    if (is_wp_error($result)) {
        wp_send_json_error(array(
            'message' => $result->get_error_message(),
            $result->get_error_data()['status'] ?? 500);
    )
    }

    // Store sessionId in a transient (expires in 15 minutes)
    $transient_key = 'otp_session_' . wp_create_nonce($phone_number);
    set_transient($transient_key, $result['sessionId'], 15 * MINUTE_IN_SECONDS);
}

```

```

        wp_send_json_success(array(
            'expires_at' => $result['expiresAt'],
            'transient_key' => $transient_key,
        ));
    }
    add_action('wp_ajax_nopriv_send_otp', 'authenticator_ajax_send_otp');
    add_action('wp_ajax_send_otp', 'authenticator_ajax_send_otp');

    /**
     * AJAX handler: Verify OTP.
     * Hook: wp_ajax_nopriv_verify_otp, wp_ajax_verify_otp
     */
    function authenticator_ajax_verify_otp() {
        check_ajax_referer('otp_nonce', 'nonce');

        $transient_key = sanitize_text_field($_POST['transient_key'] ?? '');
        $otp = sanitize_text_field($_POST['otp'] ?? '');

        if (empty($transient_key) || empty($otp)) {
            wp_send_json_error(array('message' => 'Missing required fields'),
        }

        $session_id = get_transient($transient_key);

        if (! $session_id) {
            wp_send_json_error(array('message' => 'Session expired. Please re
        }

        $result = authenticator_verify_otp($session_id, $otp);

        if (is_wp_error($result)) {
            wp_send_json_error(array(
                'message' => $result->get_error_message(),
            ), $result->get_error_data()['status'] ?? 500);
        }

        if ($result['verified']) {
            delete_transient($transient_key);
            wp_send_json_success(array(
                'phone_number' => $result['phoneNumber'],
            ));
        }

        wp_send_json_error(array(
            'message' => 'Invalid code. Please try again.',
            'reason' => $result['reason'] ?? 'mismatch',

```

```

    ));
}
add_action('wp_ajax_nopriv_verify_otp', 'authenticator_ajax_verify_otp');
add_action('wp_ajax_verify_otp', 'authenticator_ajax_verify_otp');

```

## Python (Flask/FastAPI)

 **FLAG:** Install requests for Flask or use httpx for FastAPI. Store credentials in environment variables.

### Flask

```

# services/authenticator.py

import os
import requests

BASE_URL = os.environ["AUTHENTICATOR_BASE_URL"]
APP_ID = os.environ["AUTHENTICATOR_APP_ID"]
APP_SECRET = os.environ["AUTHENTICATOR_APP_SECRET"]

def send_otp(phone_number: str) -> dict:
    """
    Send an OTP to a phone number.

    Args:
        phone_number: Phone number in E.164 format (e.g., +8801712345678)

    Returns:
        dict with sessionId and expiresAt

    Raises:
        Exception: If the API call fails
    """
    response = requests.post(
        f"{BASE_URL}/sendOtp",
        json={
            "appId": APP_ID,
            "appSecret": APP_SECRET,

```

```

        "phoneNumber": phone_number,
    },
    timeout=30,
)

if not response.ok:
    error = response.json()
    raise Exception(error.get("message", error.get("error", "Failed to

return response.json()

def verify_otp(session_id: str, otp: str) -> dict:
    """
    Verify an OTP code.

    Args:
        session_id: Session ID from send_otp
        otp: 6-digit OTP code

    Returns:
        dict with verified (bool) and optionally phoneNumber or reason
    """
    response = requests.post(
        f"{BASE_URL}/verifyOtp",
        json={
            "appId": APP_ID,
            "appSecret": APP_SECRET,
            "sessionId": session_id,
            "otp": otp,
        },
        timeout=30,
    )

    return response.json()

```

```
# app.py (Flask)
```

```

import os
from flask import Flask, request, jsonify, session
from services.authenticator import send_otp, verify_otp

app = Flask(__name__)
app.secret_key = os.environ["FLASK_SECRET_KEY"]

```



```
@app.route("/api/auth/send-otp", methods=["POST"])
def handle_send_otp():
    """Send an OTP to the given phone number."""
    data = request.get_json()
    phone_number = data.get("phoneNumber") if data else None

    if not phone_number:
        return jsonify({"error": "Phone number is required"}), 400

    try:
        result = send_otp(phone_number)
        session["otp_session_id"] = result["sessionId"]
        return jsonify(result)
    except Exception as e:
        return jsonify({"error": str(e)}), 500

@app.route("/api/auth/verify-otp", methods=["POST"])
def handle_verify_otp():
    """Verify the OTP code."""
    data = request.get_json()
    otp = data.get("otp") if data else None

    if not otp:
        return jsonify({"error": "OTP is required"}), 400

    session_id = session.get("otp_session_id")
    if not session_id:
        return jsonify({"error": "No active session. Please request a new"}

    result = verify_otp(session_id, otp)

    if result.get("verified"):
        session.pop("otp_session_id", None)
        return jsonify({"success": True, "phoneNumber": result["phoneNumber"]})

    return jsonify({"success": False, "reason": result.get("reason")}), 200

if __name__ == "__main__":
    app.run(debug=True)
```

## FastAPI

```
# main.py (FastAPI)

import os
from typing import Optional

import httpx
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, field_validator

app = FastAPI()

BASE_URL = os.environ["AUTHENTICATOR_BASE_URL"]
APP_ID = os.environ["AUTHENTICATOR_APP_ID"]
APP_SECRET = os.environ["AUTHENTICATOR_APP_SECRET"]

# In production, use Redis or a database for session storage
session_store: dict[str, str] = {}

class SendOtpRequest(BaseModel):
    phoneNumber: str

    @field_validator("phoneNumber")
    @classmethod
    def validate_phone(cls, v: str) -> str:
        import re
        if not re.match(r"^\+[1-9]\d{1,14}$", v):
            raise ValueError("Phone number must be in E.164 format")
        return v

class VerifyOtpRequest(BaseModel):
    otp: str

    @field_validator("otp")
    @classmethod
    def validate_otp(cls, v: str) -> str:
        if not v.isdigit() or len(v) != 6:
            raise ValueError("OTP must be exactly 6 digits")
        return v

@app.post("/api/auth/send-otp")
```

```
async def handle_send_otp(req: SendOtpRequest):
    """Send an OTP to the given phone number."""
    async with httpx.AsyncClient() as client:
        response = await client.post(
            f"{BASE_URL}/sendOtp",
            json={
                "appId": APP_ID,
                "appSecret": APP_SECRET,
                "phoneNumber": req.phoneNumber,
            },
            timeout=30.0,
        )

        if response.status_code != 200:
            error = response.json()
            raise HTTPException(
                status_code=response.status_code,
                detail=error.get("message", "Failed to send OTP"),
            )

        result = response.json()
        return result

@app.post("/api/auth/verify-otp")
async def handle_verify_otp(req: VerifyOtpRequest):
    """Verify the OTP code."""
    # In production, read sessionId from the user's session/token
    # This example uses a simple store – replace with your session mechan
    session_id = session_store.get("current")

    if not session_id:
        raise HTTPException(
            status_code=400,
            detail="No active session. Please request a new code.",
        )

    async with httpx.AsyncClient() as client:
        response = await client.post(
            f"{BASE_URL}/verifyOtp",
            json={
                "appId": APP_ID,
                "appSecret": APP_SECRET,
                "sessionId": session_id,
                "otp": req.otp,
            },
        )
```

```

        timeout=30.0,
    )

    result = response.json()

    if result.get("verified"):
        return {"success": True, "phoneNumber": result["phoneNumber"]}

    return {"success": False, "reason": result.get("reason")}

```

## Dart (Server-Side)

 **FLAG:** This is for Dart server-side (e.g., Dart Frog, Shelf). Do not embed appSecret in Flutter client apps.

```

// lib/services/authenticator_service.dart

import 'dart:convert';
import 'package:http/http.dart' as http;

/// Authenticator API service for Dart server-side applications.
class AuthenticatorService {
    final String baseUrl;
    final String appId;
    final String appSecret;

    AuthenticatorService({
        required this.baseUrl,
        required this.appId,
        required this.appSecret,
    });

    /// Send an OTP to a phone number.
    ///
    /// [phoneNumber] must be in E.164 format (e.g., +8801712345678).
    /// Returns a map with sessionId and expiresAt.
    Future<Map<String, dynamic>> sendOtp(String phoneNumber) async {
        final response = await http.post(
            Uri.parse('$baseUrl/sendOtp'),
            headers: {'Content-Type': 'application/json'},

```

```

        body: jsonEncode({
            'appId': appId,
            'appSecret': appSecret,
            'phoneNumber': phoneNumber,
        }),
    );

    final data = jsonDecode(response.body) as Map<String, dynamic>;

    if (response.statusCode != 200) {
        throw Exception(data['message'] ?? data['error'] ?? 'Failed to send')
    }

    return data;
}

/// Verify an OTP code.
///
/// [sessionId] is the session ID returned by sendOtp.
/// [otp] is the 6-digit code entered by the user.
Future<Map<String, dynamic>> verifyOtp(String sessionId, String otp) async {
    final response = await http.post(
        Uri.parse('$baseUrl/verifyOtp'),
        headers: {'Content-Type': 'application/json'},
        body: jsonEncode({
            'appId': appId,
            'appSecret': appSecret,
            'sessionId': sessionId,
            'otp': otp,
        }),
    );

    return jsonDecode(response.body) as Map<String, dynamic>;
}

/// Check OTP status or resend on failure.
///
/// [sessionId] is the session ID from sendOtp.
/// [resend] set to true to resend if the original SMS failed.
Future<Map<String, dynamic>> checkOtpStatus(
    String sessionId, {
    bool resend = false,
}) async {
    final response = await http.post(
        Uri.parse('$baseUrl/otpStatus'),
        headers: {'Content-Type': 'application/json'},

```

```

        body: jsonEncode({
          'appId': appId,
          'sessionId': sessionId,
          'resend': resend,
        }),
      );

      return jsonDecode(response.body) as Map<String, dynamic>;
    }
  }
}

```

## Usage with Dart Frog

```

// routes/api/auth/send_otp.dart

import 'dart:io';
import 'package:dart_frog/dart_frog.dart';
import '../lib/services/authenticator_service.dart';

final authenticator = AuthenticatorService(
  baseUrl: Platform.environment['AUTHENTICATOR_BASE_URL']!,
  appId: Platform.environment['AUTHENTICATOR_APP_ID']!,
  appSecret: Platform.environment['AUTHENTICATOR_APP_SECRET']!,
);

Future<Response> onRequest(RequestContext context) async {
  if (context.request.method != HttpMethod.post) {
    return Response.json(statusCode: 405, body: {'error': 'method_not_allowed'});
  }

  final body = await context.request.json() as Map<String, dynamic>;
  final phoneNumber = body['phoneNumber'] as String?;

  if (phoneNumber == null || phoneNumber.isEmpty) {
    return Response.json(
      statusCode: 400,
      body: {'error': 'Phone number is required'},
    );
  }

  try {
    final result = await authenticator.sendOtp(phoneNumber);
    return Response.json(body: result);
  } catch (e) {
    return Response.json(
      statusCode: 500,
      body: {'error': e.toString()},
    );
  }
}

```

```
return Response.json(  
    statusCode: 500,  
    body: {'error': e.toString()},  
);  
}  
}
```

## WordPress Plugin

Save this as wp-content/plugins/authenticator-otp/authenticator-otp.php:

```
<?php  
/**  
 * Plugin Name: Authenticator OTP Verification  
 * Plugin URI: https://authenticator-15fb7.web.app  
 * Description: Add SMS OTP verification to your WordPress site using the  
 * Version: 1.0.0  
 * Author: Authenticator  
 * License: Proprietary  
 *  
 * Requires at least: 5.0  
 * Requires PHP: 7.4  
 */  
  
if (! defined('ABSPATH')) {  
    exit;  
}  
  
// — Settings Page —————  
  
/**  
 * Register the settings menu page.  
 */  
function authenticator_otp_admin_menu() {  
    add_options_page(  
        'Authenticator OTP Settings',  
        'Authenticator OTP',  
        'manage_options',  
        'authenticator-otp',  
        'authenticator_otp_settings_page',  
    );  
}
```

```

    );
}
add_action('admin_menu', 'authenticator_otp_admin_menu');

/**
 * Register settings.
 */
function authenticator_otp_register_settings() {
    register_setting('authenticator_otp_group', 'authenticator_base_url',
        'type' => 'string',
        'sanitize_callback' => 'esc_url_raw',
        'default' => 'https://asia-southeast1-authenticator-15fb7.cloudfu
    ));

    register_setting('authenticator_otp_group', 'authenticator_app_id', a
        'type' => 'string',
        'sanitize_callback' => 'sanitize_text_field',
    ));

    register_setting('authenticator_otp_group', 'authenticator_app_secret
        'type' => 'string',
        'sanitize_callback' => 'sanitize_text_field',
    ));
}
add_action('admin_init', 'authenticator_otp_register_settings');

/**
 * Render the settings page.
 */
function authenticator_otp_settings_page() {
    if (! current_user_can('manage_options')) {
        return;
    }
    ?>
    <div class="wrap">
        <h1>Authenticator OTP Settings</h1>
        <form method="post" action="options.php">
            <?php settings_fields('authenticator_otp_group'); ?>
            <table class="form-table">
                <tr>
                    <th scope="row"><label for="authenticator_base_url">B
                    <td>
                        <input
                            type="url"
                            id="authenticator_base_url"
                            name="authenticator_base_url"

```



```

        value="<?php echo esc_attr(get_option('authen
        class="regular-text"

        />
        <p class="description">Do not change unless instr
    </td>
</tr>
<tr>
    <th scope="row"><label for="authenticator_app_id">App
    <td>
        <input
            type="text"
            id="authenticator_app_id"
            name="authenticator_app_id"
            value="<?php echo esc_attr(get_option('authen
            class="regular-text"
            required

        />
        <p class="description">From the Authenticator das
    </td>
</tr>
<tr>
    <th scope="row"><label for="authenticator_app_secret"
    <td>
        <input
            type="password"
            id="authenticator_app_secret"
            name="authenticator_app_secret"
            value="<?php echo esc_attr(get_option('authen
            class="regular-text"
            required

        />
        <p class="description">Keep this secret. Never sh
    </td>
</tr>
</table>
<?php submit_button(); ?>
</form>
</div>
<?php
}

// — API Service Functions —————

/**
 * Get the API base URL.
 *

```

```

* @return string
*/
function authenticator_get_base_url(): string {
    return get_option(
        'authenticator_base_url',
        'https://asia-southeast1-authenticator-15fb7.cloudfunctions.net'
    );
}

/**
 * Send an OTP to a phone number.
 *
 * @param string $phone_number Phone number in E.164 format
 * @return array|WP_Error
 */
function authenticator_send_otp($phone_number) {
    $base_url = authenticator_get_base_url();
    $app_id = get_option('authenticator_app_id');
    $app_secret = get_option('authenticator_app_secret');

    if (empty($app_id) || empty($app_secret)) {
        return new WP_Error('not_configured', 'Authenticator is not configured');
    }

    $response = wp_remote_post($base_url . '/sendOtp', array(
        'headers' => array('Content-Type' => 'application/json'),
        'body' => json_encode(array(
            'appId' => $app_id,
            'appSecret' => $app_secret,
            'phoneNumber' => $phone_number,
        )),
        'timeout' => 30,
    ));

    if (is_wp_error($response)) {
        return $response;
    }

    $body = json_decode(wp_remote_retrieve_body($response), true);
    $code = wp_remote_retrieve_response_code($response);

    if ($code !== 200) {
        return new WP_Error(
            $body['error'] ?? 'unknown',
            $body['message'] ?? 'Failed to send OTP',
            array('status' => $code)
        );
    }
}

```

```

    );
}

return $body;
}

/**
 * Verify an OTP code.
 *
 * @param string $session_id Session ID from authenticator_send_otp
 * @param string $otp        6-digit OTP code
 * @return array|WP_Error
 */
function authenticator_verify_otp($session_id, $otp) {
    $base_url = authenticator_get_base_url();
    $app_id = get_option('authenticator_app_id');
    $app_secret = get_option('authenticator_app_secret');

    if (empty($app_id) || empty($app_secret)) {
        return new WP_Error('not_configured', 'Authenticator is not confi
    }

    $response = wp_remote_post($base_url . '/verifyOtp', array(
        'headers' => array('Content-Type' => 'application/json'),
        'body' => json_encode(array(
            'appId' => $app_id,
            'appSecret' => $app_secret,
            'sessionId' => $session_id,
            'otp' => $otp,
        )),
        'timeout' => 30,
    ));

    if (is_wp_error($response)) {
        return $response;
    }

    $body = json_decode(wp_remote_retrieve_body($response), true);
    $code = wp_remote_retrieve_response_code($response);

    if ($code !== 200) {
        return new WP_Error(
            $body['error'] ?? 'unknown',
            $body['message'] ?? 'Failed to verify OTP',
            array('status' => $code)
        );
    }
}

```

```

    }

    return $body;
}

// ——— AJAX Handlers —————

/**
 * AJAX handler for sending OTP.
 */
function authenticator_ajax_send_otp() {
    check_ajax_referer('authenticator_otp_nonce', 'nonce');

    $phone = sanitize_text_field($_POST['phone_number'] ?? '');

    if (empty($phone)) {
        wp_send_json_error(array('message' => 'Phone number is required'))
    }

    $result = authenticator_send_otp($phone);

    if (is_wp_error($result)) {
        wp_send_json_error(array(
            'message' => $result->get_error_message(),
        ));
    }

    // Store sessionId in a transient (15-minute expiry)
    $key = 'auth_otp_' . wp_create_nonce($phone . time());
    set_transient($key, $result['sessionId'], 15 * MINUTE_IN_SECONDS);

    wp_send_json_success(array(
        'expires_at' => $result['expiresAt'],
        'key' => $key,
    ));
}

add_action('wp_ajax_nopriv_authenticator_send_otp', 'authenticator_ajax_send_otp');
add_action('wp_ajax_authenticator_send_otp', 'authenticator_ajax_send_otp');

/**
 * AJAX handler for verifying OTP.
 */
function authenticator_ajax_verify_otp() {
    check_ajax_referer('authenticator_otp_nonce', 'nonce');

    $key = sanitize_text_field($_POST['key'] ?? '');

```

```

$otp = sanitize_text_field($_POST['otp'] ?? '');

if (empty($key) || empty($otp)) {
    wp_send_json_error(array('message' => 'Missing required fields'),
}

$session_id = get_transient($key);

if (! $session_id) {
    wp_send_json_error(array('message' => 'Session expired. Please re
}

$result = authenticator_verify_otp($session_id, $otp);

if (is_wp_error($result)) {
    wp_send_json_error(array('message' => $result->get_error_message(
}

if (! empty($result['verified'])) {
    delete_transient($key);
    wp_send_json_success(array('phone_number' => $result['phoneNumber
}

wp_send_json_error(array(
    'message' => 'Invalid code. Please try again.',
    'reason' => $result['reason'] ?? 'mismatch',
));
}
add_action('wp_ajax_nopriv_authenticator_verify_otp', 'authenticator_ajax
add_action('wp_ajax_authenticator_verify_otp', 'authenticator_ajax_verify

// — Shortcode —————

/**
 * Shortcode: [authenticator_otp_form]
 *
 * Renders a complete OTP verification form with AJAX handling.
 *
 * Attributes:
 * - redirect (string) URL to redirect after successful verification
 * - button_text (string) Text for the submit button (default: "Verify"
 */
function authenticator_otp_shortcode($atts) {
    $atts = shortcode_atts(array(
        'redirect' => '',
        'button_text' => 'Verify',

```

```

), $atts, 'authenticator_otp_form');

ob_start();
?>
<div id="authenticator-otp-form">
  <!-- Step 1: Phone Number -->
  <div id="otp-step-phone">
    <label for="otp-phone">Phone Number:</label>
    <input
      type="tel"
      id="otp-phone"
      placeholder="+8801712345678"
      pattern="^\+[1-9]\d{1,14}$"
      required
    />
    <button type="button" id="otp-send-btn" class="button">
      Send Code
    </button>
  </div>

  <!-- Step 2: OTP Code -->
  <div id="otp-step-verify" style="display: none;">
    <label for="otp-code">Verification Code:</label>
    <input
      type="text"
      id="otp-code"
      placeholder="123456"
      maxlength="6"
      pattern="\d{6}"
      required
    />
    <button type="button" id="otp-verify-btn" class="button">
      <?php echo esc_html($atts['button_text']); ?>
    </button>
  </div>

  <!-- Messages -->
  <p id="otp-message" style="display: none;"></p>
  <p id="otp-error" style="display: none; color: red;"></p>
</div>

<script>
jQuery(document).ready(function($) {
  var otpKey = '';
  var nonce = '<?php echo wp_create_nonce("authenticator_otp_nonce"

```

```

$('#otp-send-btn').on('click', function() {
    var phone = $('#otp-phone').val().trim();
    if (!phone) { return; }

    $(this).prop('disabled', true).text('Sending...');
    $('#otp-error').hide();

    $.post.ajaxurl, {
        action: 'authenticator_send_otp',
        nonce: nonce,
        phone_number: phone,
    }, function(response) {
        if (response.success) {
            otpKey = response.data.key;
            $('#otp-step-phone').hide();
            $('#otp-step-verify').show();
            $('#otp-message').text('Code sent!').show();
        } else {
            $('#otp-error').text(response.data.message).show();
            $('#otp-send-btn').prop('disabled', false).text('Send');
        }
    });
});

$('#otp-verify-btn').on('click', function() {
    var otp = $('#otp-code').val().trim();
    if (!otp) { return; }

    $(this).prop('disabled', true).text('Verifying...');
    $('#otp-error').hide();

    $.post.ajaxurl, {
        action: 'authenticator_verify_otp',
        nonce: nonce,
        key: otpKey,
        otp: otp,
    }, function(response) {
        if (response.success) {
            $('#otp-message').text('Verified!').show();
            <?php if (! empty($atts['redirect'])) : ?>
                window.location.href = '<?php echo esc_js($atts['
            <?php endif; ?>
        } else {
            $('#otp-error').text(response.data.message).show();
            $('#otp-verify-btn').prop('disabled', false).text('<?|
        }
    }

```

```
        });  
    });  
});  
</script>  
<?php  
    return ob_get_clean();  
}  
add_shortcode('authenticator_otp_form', 'authenticator_otp_shortcode');
```

## Webhooks

Authenticator can notify your server when the OTP delivery status changes. Configure your webhook URL in the app dashboard.

### Webhook Events

| EVENT   | TRIGGER                                     |
|---------|---------------------------------------------|
| sent    | SMS was successfully delivered to the phone |
| failed  | SMS delivery failed                         |
| expired | OTP session expired without being verified  |

### Webhook Payload

```
{  
  "event": "sent",  
  "sessionId": "f47ac10b-58cc-4372-a567-0e02b2c3d479",  
  "appId": "app_abc123",  
  "timestamp": 1720000000000,  
  "data": {}  
}
```

### Verifying Webhooks



Webhooks include a signature for verification. Check the X-Webhook-Signature header against your webhook secret to verify authenticity.

---

## FAQ

### General

**Q: Which countries do you support?**

A: Bangladesh (+880) is fully supported. Additional countries are available on request — contact us through the dashboard.

**Q: What is the OTP format?**

A: 6-digit numeric code (e.g., 384726), generated using a cryptographically secure random number generator.

**Q: How long is the OTP valid?**

A: 10 minutes from the time it was sent. After that, the session expires and the user must request a new OTP.

**Q: How many verification attempts are allowed?**

A: 3 attempts per session. After 3 wrong guesses, the session is locked and the user must request a new OTP.

### Rate Limits

**Q: How many OTPs can I send to one phone number?**

A: 3 per 10 minutes (default, configurable per app). There is also a 30-second cooldown between OTPs to the same number.

**Q: Can I increase the rate limit?**

A: Yes — contact support through the dashboard with your use case, and we can adjust your app's rate limits.

**Q: What happens if I hit the rate limit?**

A: You receive a 429 response with a Retry-After header telling you how many seconds to wait.

### Integration

**Q: Do I need to send the OTP to the user myself?**

A: No. When you call `send0tp`, we generate the OTP and send the SMS automatically. You just collect the code from the user and call `verify0tp`.

**Q: Can I customize the SMS message?**

A: Yes. Set a custom SMS template in your app settings on the dashboard. Use `{appName}`, `{otp}`, and `{ttl}` placeholders.

**Q: Is there a test/sandbox mode?**

A: We use the same production endpoints for testing. Purchase a small credit package and test with real phone numbers.

**Q: What happens if the SMS fails to deliver?**

A: Use the `otpStatus` endpoint with `resend: true` to trigger a resend. The service will generate a new OTP and re-queue the SMS. Maximum 2 resends per session.

## Security

**Q: Can I call the API from my frontend (React, Flutter, etc.)?**

A: **No.** Your `appSecret` must never be exposed in client-side code. Always call the API from your backend server and proxy requests from your frontend.

**Q: How should I store my credentials?**

A: Use environment variables (`AUTHENTICATOR_APP_ID`, `AUTHENTICATOR_APP_SECRET`) or a secure secrets manager. Never hardcode them in source code or commit them to Git.

## Changelog

| VERSION | DATE    | CHANGES                                                                                                                                                          |
|---------|---------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 5.0.0   | 2025-07 | Added <code>send0tp</code> , <code>verify0tp</code> , <code>otpStatus</code> endpoints. App-based authentication. Credit system. Webhooks. Custom SMS templates. |
| 4.0.0   | 2025-01 | Initial authenticator device-based verification ( <code>startVerification</code> , <code>checkAuth</code> ).                                                     |

## Support

- **Dashboard:** <https://authenticator-15fb7.web.app>
- **API Status:** Check the dashboard for real-time status
- **Documentation:** This document is the canonical API reference

---

© 2025 Authenticator. All rights reserved.



Reliable OTP verification and bulk SMS for apps in Bangladesh and beyond. Simple API, instant delivery, developer-friendly.

### Product

[Pricing](#)

[Documentation](#)

[Contact](#)

### Company

[About](#)

[Privacy Policy](#)

[Terms of Service](#)

### Support

[support@dprelay.com](mailto:support@dprelay.com)

+880 1700-000000

---

© 2026 [dpRelay](#). All rights reserved. Built for Bangladesh.